

Difusão viscosa com diferenças finitas e OpenMP

Matheus Marinho

20 de setembro de 2026

O modelo

O enunciado pede só o efeito da viscosidade. Tirando advecção, pressão e forças externas de Navier-Stokes, cada componente da velocidade obedece a uma equação de difusão, e as duas ficam independentes:

$$\partial u / \partial t = \nu \nabla^2 u \quad \partial v / \partial t = \nu \nabla^2 v$$

O domínio é $[0, 1] \times [0, 1]$ em uma malha $n \times n$ com $h = 1/n$. As derivadas segundas usam diferenças centrais e o tempo avança por Euler explícito:

$$\nabla^2 u \approx (u_{\text{dir}} + u_{\text{esq}} + u_{\text{cima}} + u_{\text{baixo}} - 4 \cdot u_{\text{centro}}) / h^2 \quad u^{\text{novo}} = u^{\text{atual}} + \nu \cdot \Delta t \cdot \nabla^2 u^{\text{atual}}$$

O laplaciano compara a célula com a vizinhança. Se ela está acima, o valor cai; se está abaixo, sobe. É isso que espalha uma perturbação e uniformiza o campo.

Euler explícito em duas dimensões só é estável se $r = \nu \cdot \Delta t / h^2 \leq 1/4$. O programa define Δt a partir de $r = 0,20$. A leitura prática: mais viscosidade significa difusão mais rápida e passo de tempo **menor** — aumentar ν não permite aumentar Δt .

Como o código está montado

Quatro matrizes. u , v são o estado atual; un , vn o próximo. O novo valor é calculado lendo só o estado atual. Atualizar no mesmo vetor faria o resultado depender da ordem de varredura, porque uma célula já atualizada entraria como vizinha da seguinte — e em paralelo seria condição de corrida. Ao fim do passo só os ponteiros são trocados, sem cópia de dados.

Bordas periódicas. O vizinho da última coluna é a primeira, via resto da divisão. Uma parede seria uma condição de contorno que o enunciado não pede, e faria o campo constante mudar justamente na borda, estragando a validação. Em troca ganha-se um teste: com o contorno fechado sobre si mesmo, a soma de todas as células tem que se conservar.

Como cada célula do passo novo depende apenas de valores que ninguém altera durante o passo, não há dependência entre células. É por isso que o laço paraleliza sem **critical**, **atomic** ou **reduction**.

Validação

Malha 1024×1024 , 200 passos, sequencial.

Teste	Esperado	Obtido
fluido parado ($u=0$, $v=0$)	continua zero	maior `
campo constante ($u=2$, $v=-1$)	não muda	desvio = 0,000e+00
perturbação central	pico cai e difunde	2,000000 → 1,001986
soma total	conservada	1048577,0000 → 1048577,0000 (erro 1,5e-08)
$r = 0,30$, acima de 1/4	deve divergir	pico final 3,1e+26

Os dois primeiros dão zero **exato**, não aproximado: num campo uniforme a conta $4x - 4x$ é exatamente zero em ponto flutuante. O erro de 1,5e-08 sobre uma soma de 1,05e+06 é arredondamento (1,5e-14 relativo), não perda de massa. E o último teste confirma o limite de estabilidade pelo lado de fora.

Paralelização

São sete funções com **exatamente o mesmo corpo**; só a diretiva muda. O experimento só mede o que promete se a diretiva for a única variável.

```
static void passo_static(const Malha *m, const double *u, const double *v,
                        double *un, double *vn)
{
#pragma omp parallel for schedule(static)
```

```

for (int i = 0; i < m->n; i++)
    for (int j = 0; j < m->n; j++)
        atualiza_celula(m, u, v, un, vn, i, j);
}

```

As seis diretivas comparadas:

```

#pragma omp parallel for schedule(static)
#pragma omp parallel for collapse(2) schedule(static)
#pragma omp parallel for schedule(dynamic)
#pragma omp parallel for collapse(2) schedule(dynamic)
#pragma omp parallel for schedule(guided)
#pragma omp parallel for collapse(2) schedule(guided)

```

Nenhuma recebeu tamanho de bloco explícito, então vale o padrão: bloco 1 para `dynamic`, bloco decrescente para `guided`. Esse detalhe explica a maior parte dos resultados. `collapse(2)` funde os dois laços num espaço de 1.048.576 iterações em vez de 1.024 — não muda a quantidade de contas, só o que o OpenMP tem para distribuir.

As seis versões produziram o campo final idêntico **byte a byte** ao sequencial (`memcpy`, sem tolerância), em todas as contagens de thread.

Ambiente e resultados

NPAD/UFRN, partição `amd-512`, nó `r2n00` com `--exclusive: 2 sockets × 64 núcleos`, 2 threads por núcleo, 512 GB. gcc 8.5.0 com `-O2 -Wall -Wextra -std=c11 -fopenmp, OMP_PLACES=cores, OMP_PROC_BIND=close`. Tempo por `clock_gettime(CLOCK_MONOTONIC)`, mediana de 5 execuções.

O nó exclusivo e a fixação das threads existem para que a diretiva seja a única coisa que varia entre as medições. O tempo sequencial saiu 0,6973, 0,6977 e 0,6974 s nas três rodadas, o que confirma a estabilidade da medida.

Malha 1024×1024 , 200 passos. Tempo em segundos, speedup entre parênteses.

schedule	collapse	16 threads	32 threads	64 threads
sequencial	—	0,6973	0,6977	0,6974
<code>static</code>	não	0,0567 (12,3×)	0,0315 (22,1×)	0,0218 (32,1×)
<code>static</code>	<code>collapse(2)</code>	0,1068 (6,5×)	0,0558 (12,5×)	0,0323 (21,6×)
<code>dynamic</code>	não	0,2290 (3,0×)	0,2183 (3,2×)	0,2866 (2,4×)
<code>dynamic</code>	<code>collapse(2)</code>	7,4154 (0,09×)	6,9383 (0,10×)	7,0872 (0,10×)
<code>guided</code>	não	0,1796 (3,9×)	0,1461 (4,8×)	0,1397 (5,0×)
<code>guided</code>	<code>collapse(2)</code>	0,1925 (3,6×)	0,1502 (4,7×)	0,1476 (4,7×)

Análise

static vence em todas as configurações. Toda célula custa o mesmo: cinco leituras, uma multiplicação e uma soma por componente. Não há desbalanceamento para corrigir, então não há nada que justifique pagar por distribuição dinâmica. `static` divide as linhas antes de começar e depois nenhuma thread consulta nada compartilhado.

dynamic paga o contador do laço. Cada bloco entregue exige um acesso sincronizado. O revelador é que `dynamic` não melhora com mais threads: 0,2290 s com 16 e 0,2866 s com 64, ou seja, termina pior do que começou. Mais threads disputam o mesmo contador, e o trabalho por bloco é pequeno demais para esconder o custo. `guided` fica sempre à frente do `dynamic`, porque começa com blocos grandes e faz muito menos entregas, mas empaca em torno de 5× e continua consultando um contador que `static` não consulta.

Repetindo o experimento inteiro, os tempos de `static` se reproduziram dentro de 1%, enquanto `dynamic` e `guided` variaram bem mais em 64 threads — `guided` chegou a 0,2345 s numa repetição contra 0,1397 s nesta. Essa instabilidade é ela própria um resultado: o custo dessas políticas depende de disputa em tempo de execução, que muda de uma execução para outra. O de `static` não depende.

dynamic com collapse(2) é o pior caso. O espaço passa a 1.048.576 iterações com bloco 1: cerca de 210 milhões de entregas sincronizadas na simulação, cada uma para calcular **uma célula**. Dá 7 s contra 0,022 s da melhor versão — 10 vezes mais lento que o código sequencial. É a demonstração mais direta de que uma cláusula pode custar muito mais do que o cálculo que distribui.

collapse(2) só atrapalha aqui. Ele serve quando o laço externo tem iterações de menos para ocupar as threads. Não é o caso: 1.024 linhas para no máximo 64 threads, 16 linhas por thread. O que ele acrescenta é custo de indexação — recuperar

`i` e `j` de um índice linear a cada iteração, e perder o laço interno simples que o compilador otimiza bem. Com `static`, custou de 1,5× a 1,9× de tempo a mais.

O speedup é sublinear, e isso é esperado. Com `static`, a eficiência cai de 77% (16 threads) para 69% (32) e 50% (64). O problema é limitado por memória: a cada passo o programa percorre quatro matrizes de 8 MB, muito além do cache, com poucas operações por byte lido. Estimando pela via compulsória — 32 MB por passo, 200 passos, 6,4 GB em 0,022 s — dá cerca de 290 GB/s, compatível com o limite de um nó de dois sockets. Some-se a isso um gargalo que **não foi corrigido de propósito**: as matrizes são inicializadas sequencialmente, então por *first touch* quase toda a memória fica perto de um socket só e as threads do outro pagam acesso remoto. Inicializar em paralelo distribuiria as páginas, mas otimizar o laço mediria a otimização, não as cláusulas, que é o que o enunciado manda explorar.

Código

difusao_viscosa.c · 353 linhas

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <math.h>
4 #include <omp.h>
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <string.h>
8 #include <time.h>
9
10 typedef struct {
11     int n;
12     double h;
13     double viscosidade;
14     double dt;
15 } Malha;
16
17 typedef void (*Passo)(const Malha *, const double *, const double *,
18                     double *, double *);
19
20 typedef struct {
21     const char *schedule;
22     const char *collapse;
23     Passo funcao;
24     double segundos;
25     int confere;
26 } Versao;
27
28 static double agora(void)
29 {
30     struct timespec t;
31     clock_gettime(CLOCK_MONOTONIC, &t);
32     return (double)t.tv_sec + (double)t.tv_nsec * 1e-9;
33 }
34
35 static double *aloca(int n)
36 {
37     double *p = malloc((size_t)n * (size_t)n * sizeof(double));
38     if (p == NULL) {
39         fprintf(stderr, "memoria insuficiente\n");
40         exit(EXIT_FAILURE);
41     }
42     return p;
43 }
44
45 static inline void atualiza_celula(const Malha *m,
46                                 const double *u, const double *v,
47                                 double *un, double *vn, int i, int j)
48 {
49     const int n = m->n;
50     const int i_ant = (i + n - 1) % n;
51     const int i_prox = (i + 1) % n;
52     const int j_ant = (j + n - 1) % n;
53     const int j_prox = (j + 1) % n;
54     const int centro = i * n + j;
55     const double fator = m->viscosidade * m->dt / (m->h * m->h);
56
57     const double laplaciano_u = u[i_prox * n + j] + u[i_ant * n + j]
58                               + u[i * n + j_prox] + u[i * n + j_ant]
59                               - 4.0 * u[centro];
60     const double laplaciano_v = v[i_prox * n + j] + v[i_ant * n + j]
61                               + v[i * n + j_prox] + v[i * n + j_ant]
62                               - 4.0 * v[centro];
63
64     un[centro] = u[centro] + fator * laplaciano_u;
65     vn[centro] = v[centro] + fator * laplaciano_v;
66 }
67
68 static void passo_sequencial(const Malha *m, const double *u, const double *v,
69                             double *un, double *vn)
```

```

70 {
71     for (int i = 0; i < m->n; i++)
72         for (int j = 0; j < m->n; j++)
73             atualiza_celula(m, u, v, un, vn, i, j);
74 }
75
76 static void passo_static(const Malha *m, const double *u, const double *v,
77                         double *un, double *vn)
78 {
79     #pragma omp parallel for schedule(static)
80     for (int i = 0; i < m->n; i++)
81         for (int j = 0; j < m->n; j++)
82             atualiza_celula(m, u, v, un, vn, i, j);
83 }
84
85 static void passo_static_collapse(const Malha *m, const double *u, const double *v,
86                                 double *un, double *vn)
87 {
88     #pragma omp parallel for collapse(2) schedule(static)
89     for (int i = 0; i < m->n; i++)
90         for (int j = 0; j < m->n; j++)
91             atualiza_celula(m, u, v, un, vn, i, j);
92 }
93
94 static void passo_dynamic(const Malha *m, const double *u, const double *v,
95                          double *un, double *vn)
96 {
97     #pragma omp parallel for schedule(dynamic)
98     for (int i = 0; i < m->n; i++)
99         for (int j = 0; j < m->n; j++)
100             atualiza_celula(m, u, v, un, vn, i, j);
101 }
102
103 static void passo_dynamic_collapse(const Malha *m, const double *u, const double *v,
104                                   double *un, double *vn)
105 {
106     #pragma omp parallel for collapse(2) schedule(dynamic)
107     for (int i = 0; i < m->n; i++)
108         for (int j = 0; j < m->n; j++)
109             atualiza_celula(m, u, v, un, vn, i, j);
110 }
111
112 static void passo_guided(const Malha *m, const double *u, const double *v,
113                        double *un, double *vn)
114 {
115     #pragma omp parallel for schedule(guided)
116     for (int i = 0; i < m->n; i++)
117         for (int j = 0; j < m->n; j++)
118             atualiza_celula(m, u, v, un, vn, i, j);
119 }
120
121 static void passo_guided_collapse(const Malha *m, const double *u, const double *v,
122                                  double *un, double *vn)
123 {
124     #pragma omp parallel for collapse(2) schedule(guided)
125     for (int i = 0; i < m->n; i++)
126         for (int j = 0; j < m->n; j++)
127             atualiza_celula(m, u, v, un, vn, i, j);
128 }
129
130 static void simular(const Malha *m, double **u, double **v, double **un, double **vn,
131                   int passos, Passo passo)
132 {
133     for (int t = 0; t < passos; t++) {
134         passo(m, *u, *v, *un, *vn);
135         double *troca;
136         troca = *u; *u = *un; *un = troca;
137         troca = *v; *v = *vn; *vn = troca;
138     }
139 }
140
141 static void preencher(double *campo, int n, double valor)
142 {
143     for (int k = 0; k < n * n; k++)
144         campo[k] = valor;
145 }
146
147 static double maior_desvio(const double *campo, int n, double referencia)
148 {
149     double maior = 0.0;
150     for (int k = 0; k < n * n; k++) {
151         double d = fabs(campo[k] - referencia);
152         if (d > maior)
153             maior = d;
154     }
155     return maior;
156 }
157
158 static double soma(const double *campo, int n)

```

```

159 {
160     double s = 0.0;
161     for (int k = 0; k < n * n; k++)
162         s += campo[k];
163     return s;
164 }
165
166 static double maximo(const double *campo, int n)
167 {
168     double maior = campo[0];
169     for (int k = 1; k < n * n; k++)
170         if (campo[k] > maior)
171             maior = campo[k];
172     return maior;
173 }
174
175 static int comparar_double(const void *a, const void *b)
176 {
177     double x = *(const double *)a;
178     double y = *(const double *)b;
179     return (x > y) - (x < y);
180 }
181
182 static double mediana(double *v, int n)
183 {
184     qsort(v, (size_t)n, sizeof(double), comparar_double);
185     return v[n / 2];
186 }
187
188 static double passo_de_tempo(const Malha *m, double r)
189 {
190     return r * m->h * m->h / m->viscosidade;
191 }
192
193 static void teste_campo_parado(const Malha *m, int passos)
194 {
195     double *u = aloca(m->n), *v = aloca(m->n);
196     double *un = aloca(m->n), *vn = aloca(m->n);
197
198     preencher(u, m->n, 0.0);
199     preencher(v, m->n, 0.0);
200     simular(m, &u, &v, &un, &vn, passos, passo_sequencial);
201
202     printf("1) fluido parado      u=0, v=0      maior |u| = %.3e   maior |v| = %.3e\n",
203           maior_desvio(u, m->n, 0.0), maior_desvio(v, m->n, 0.0));
204
205     free(u); free(v); free(un); free(vn);
206 }
207
208 static void teste_campo_constante(const Malha *m, int passos)
209 {
210     const double u0 = 2.0, v0 = -1.0;
211     double *u = aloca(m->n), *v = aloca(m->n);
212     double *un = aloca(m->n), *vn = aloca(m->n);
213
214     preencher(u, m->n, u0);
215     preencher(v, m->n, v0);
216     simular(m, &u, &v, &un, &vn, passos, passo_sequencial);
217
218     printf("2) campo constante    u=%.1f, v=%.1f    desvio |u| = %.3e   desvio |v| = %.3e\n",
219           u0, v0, maior_desvio(u, m->n, u0), maior_desvio(v, m->n, v0));
220
221     free(u); free(v); free(un); free(vn);
222 }
223
224 static void teste_perturbacao(const Malha *m, int passos)
225 {
226     double *u = aloca(m->n), *v = aloca(m->n);
227     double *un = aloca(m->n), *vn = aloca(m->n);
228
229     preencher(u, m->n, 1.0);
230     preencher(v, m->n, 0.0);
231     u[(m->n / 2) * m->n + m->n / 2] += 1.0;
232
233     double pico_inicial = maximo(u, m->n);
234     double soma_inicial = soma(u, m->n);
235
236     simular(m, &u, &v, &un, &vn, passos, passo_sequencial);
237
238     double pico_final = maximo(u, m->n);
239     double soma_final = soma(u, m->n);
240
241     printf("3) perturbacao central pico %.6f -> %.6f   soma %.4f -> %.4f (erro %.3e)\n",
242           pico_inicial, pico_final, soma_inicial, soma_final,
243           fabs(soma_final - soma_inicial));
244
245     free(u); free(v); free(un); free(vn);
246 }
247

```

```

248 static void teste_acima_do_limite(const Malha *base, int passos)
249 {
250     const double r_instavel = 0.30;
251     Malha m = *base;
252     m.dt = passo_de_tempo(&m, r_instavel);
253
254     double *u = aloca(m.n), *v = aloca(m.n);
255     double *un = aloca(m.n), *vn = aloca(m.n);
256
257     preencher(u, m.n, 1.0);
258     preencher(v, m.n, 0.0);
259     u[(m.n / 2) * m.n + m.n / 2] += 1.0;
260
261     simular(&m, &u, &v, &un, &vn, passos, passo_sequencial);
262
263     printf("4) mesmo teste com r = %.2f (acima de 1/4)      pico final = %.3e\n",
264           r_instavel, maximo(u, m.n));
265
266     free(u); free(v); free(un); free(vn);
267 }
268
269 static double medir(const Malha *m, int passos, int execucoes, Passo passo,
270                   double *saida)
271 {
272     double *tempos = malloc((size_t)execucoes * sizeof(double));
273     double *u = aloca(m->n), *v = aloca(m->n);
274     double *un = aloca(m->n), *vn = aloca(m->n);
275
276     for (int e = 0; e < execucoes; e++) {
277         preencher(u, m->n, 1.0);
278         preencher(v, m->n, 0.0);
279         u[(m->n / 2) * m->n + m->n / 2] += 1.0;
280
281         double t0 = agora();
282         simular(m, &u, &v, &un, &vn, passos, passo);
283         tempos[e] = agora() - t0;
284     }
285
286     memcpy(saida, u, (size_t)m->n * (size_t)m->n * sizeof(double));
287
288     double t = mediana(tempos, execucoes);
289     free(tempos); free(u); free(v); free(un); free(vn);
290     return t;
291 }
292
293 int main(int argc, char *argv[])
294 {
295     int n          = (argc > 1) ? atoi(argv[1]) : 1024;
296     int passos     = (argc > 2) ? atoi(argv[2]) : 200;
297     int threads    = (argc > 3) ? atoi(argv[3]) : omp_get_max_threads();
298     int execucoes  = (argc > 4) ? atoi(argv[4]) : 5;
299
300     const double r = 0.20;
301     Malha m;
302     m.n = n;
303     m.h = 1.0 / (double)n;
304     m.viscosidade = 0.01;
305     m.dt = passo_de_tempo(&m, r);
306
307     omp_set_num_threads(threads);
308
309     printf("Malha %d x %d   h = %.3e   viscosidade = %.2f   dt = %.3e   r = %.2f\n",
310           n, n, m.h, m.viscosidade, m.dt, r);
311     printf("Passos = %d   threads = %d   execucoes = %d\n\n",
312           passos, threads, execucoes);
313
314     printf("Validacao (sequencial)\n");
315     teste_campo_parado(&m, passos);
316     teste_campo_constante(&m, passos);
317     teste_perturbacao(&m, passos);
318     teste_acima_do_limite(&m, passos);
319
320     printf("\nDesempenho (mediana de %d execucoes)\n", execucoes);
321
322     double *referencia = aloca(n);
323     double t_seq = medir(&m, passos, execucoes, passo_sequencial, referencia);
324
325     Versao versoes[6] = {
326         { "static", "nao",          passo_static,          0.0, 0 },
327         { "static", "collapse(2)",  passo_static_collapse, 0.0, 0 },
328         { "dynamic", "nao",          passo_dynamic,          0.0, 0 },
329         { "dynamic", "collapse(2)",  passo_dynamic_collapse, 0.0, 0 },
330         { "guided", "nao",           passo_guided,           0.0, 0 },
331         { "guided", "collapse(2)",  passo_guided_collapse, 0.0, 0 },
332     };
333
334     double *obtido = aloca(n);
335
336     for (int k = 0; k < 6; k++) {

```

```
337     versoes[k].segundos = medir(&m, passos, execucoes, versoes[k].funcao, obtido);
338     versoes[k].confere = (memcmp(obtido, referencia,
339                                (size_t)n * (size_t)n * sizeof(double)) == 0);
340 }
341
342 printf("%-10s %-12s %11s %9s %9s\n",
343        "schedule", "collapse", "tempo (s)", "speedup", "confere");
344 printf("%-10s %-12s %11.4f %9.2f %9s\n", "-", "-", t_seq, 1.0, "-");
345 for (int k = 0; k < 6; k++)
346     printf("%-10s %-12s %11.4f %9.2f %9s\n",
347            versoes[k].schedule, versoes[k].collapse, versoes[k].segundos,
348            t_seq / versoes[k].segundos, versoes[k].confere ? "sim" : "NAO");
349
350 free(referencia);
351 free(obtido);
352 return 0;
353 }
```